

UNIVERSIDAD DE SAN CARLOS DE GUATEMALA
FACULTAD DE INGENIERÍA
ESCUELA DE INGENIERÍA EN CIENCIAS Y SISTEMAS

Informe técnico: sistema NoSQL de inventario y ventas

Tarea2: Bases de Datos 2 NoSQL

Estudiante: Jairo Gómez

Carné: 201902672

Sección: N

Docente: Joshua Alexander Vásquez del Águila

Semestre: Segundo semestre de 2026

Fecha de entrega: 25 de septiembre de 2026

Índice

1. Introducción
2. Marco formativo
3. Objetivos
4. Tipo de base de datos NoSQL seleccionado
5. Descripción del caso de estudio
6. Uso de inteligencia artificial
7. Análisis crítico y correcciones manuales
8. Modelo implementado
9. Implementación y carga de datos
10. Operaciones CRUD
11. Consultas relevantes
12. Optimización mediante índices
13. Conclusiones
14. Reflexión individual
15. Referencias técnicas

1. Introducción

Las bases de datos NoSQL ofrecen modelos flexibles para almacenar información cuyo formato no siempre se ajusta de manera natural a tablas rígidas. En esta tarea se desarrolló un sistema de inventario y ventas para una tienda comercial mediante MongoDB, una base de datos documental. La solución administra categorías, proveedores, productos y ventas, y permite registrar información, consultar existencias, modificar productos, eliminar registros temporales y obtener indicadores útiles para la toma de decisiones.

La inteligencia artificial generativa se utilizó como apoyo durante la propuesta inicial del modelo y la elaboración de comandos. Sus resultados no se adoptaron de forma automática: se contrastaron con los requisitos del caso, se probaron en MongoDB y se ajustaron manualmente. Entre los cambios más importantes se encuentran la separación de categorías y proveedores, el uso de enteros para los valores monetarios, la inclusión de validaciones de esquema, la conservación de datos históricos dentro de cada venta y la creación de un índice compuesto.

La implementación final contiene 45 documentos iniciales distribuidos en cuatro colecciones, una demostración completa de operaciones CRUD, tres consultas relevantes y evidencia de

optimización con `explain("executionStats")`.

2. Marco formativo

2.1 Valor aplicado

Valor	Aplicación en el laboratorio
Responsabilidad	Se verificaron las propuestas generadas por IA antes de incorporarlas. Cada comando fue ejecutado y sus resultados fueron documentados mediante capturas, evitando presentar contenido no validado como si fuera correcto.

2.2 Competencias desarrolladas

Tipo de competencia	Descripción
Competencia general	Diseñar e implementar soluciones de almacenamiento de datos seleccionando el paradigma de base de datos apropiado según las necesidades de un caso real.
Competencia específica	Modelar información de inventario y ventas en MongoDB; aplicar validaciones, operaciones CRUD, consultas y agregaciones; analizar planes de ejecución; y evaluar críticamente las recomendaciones de una herramienta de IA generativa.

3. Objetivos

3.1 Objetivo general

Diseñar e implementar una base de datos documental funcional para gestionar el inventario y las ventas de una tienda, utilizando MongoDB e inteligencia artificial como herramienta de apoyo sujeta a validación técnica.

3.2 Objetivos específicos

- Modelar productos, categorías, proveedores y ventas mediante documentos BSON.
- Cargar al menos 20 registros y verificar su persistencia.
- Demostrar las operaciones de creación, lectura, actualización y eliminación.
- Ejecutar tres consultas que aporten información útil al negocio.
- Medir y mejorar el plan de ejecución de una consulta mediante un índice compuesto.

- Identificar limitaciones de la propuesta de IA y documentar las correcciones humanas realizadas.

4. Tipo de base de datos NoSQL seleccionado

Se eligió una base de datos **documental**, implementada con **MongoDB 8.0**. En este modelo, la información se almacena como documentos BSON agrupados en colecciones. Cada documento puede contener datos escalares, objetos anidados y arreglos, por lo que resulta apropiado para representar una venta completa junto con el cliente y el detalle de productos adquiridos.

MongoDB fue adecuado para este caso por las siguientes razones:

- Permite que la estructura de una venta se represente como una sola unidad lógica.
- Admite arreglos y subdocumentos para el detalle de productos.
- Facilita consultas operativas con `find()` y análisis con `aggregate()`.
- Ofrece validación mediante `$jsonSchema` sin perder la flexibilidad documental.
- Proporciona índices y estadísticas de ejecución para optimizar consultas.
- Puede desplegarse de forma reproducible mediante Docker Compose.

El diseño combina **referencias** y **datos embebidos**. Los productos referencian su categoría y proveedor mediante identificadores `ObjectId`, evitando duplicar estos catálogos. En cambio, las ventas embeben el cliente y una copia de los datos esenciales de cada producto, con el propósito de conservar el historial aunque el nombre o el precio actual del producto cambien posteriormente.

5. Descripción del caso de estudio

El escenario corresponde a una tienda que necesita controlar su catálogo, las existencias disponibles y las ventas realizadas. El sistema debe responder preguntas operativas como cuáles productos requieren reposición, qué artículos pertenecen a una categoría y cuáles han vendido más unidades.

Los procesos principales son:

1. Registrar categorías y proveedores.
2. Crear y mantener productos con SKU, precio, existencias y nivel mínimo.

3. Registrar ventas con cliente, productos, cantidades, subtotales y total.
4. Descontar existencias al preparar los datos de ventas confirmadas.
5. Consultar información útil para inventario y análisis comercial.

6. Uso de inteligencia artificial

La herramienta utilizada fue **ChatGPT de OpenAI**. Se empleó como asistente para proponer un modelo inicial y generar una primera versión de las operaciones CRUD. La propuesta fue considerada un punto de partida y luego fue revisada contra los requisitos funcionales y técnicos.

6.1 Propuesta inicial del modelo

La IA sugirió dos colecciones principales, `productos` y `ventas`, con un total de 35 documentos. La propuesta identificó correctamente las entidades centrales y recomendó que cada venta contuviera un arreglo de productos.

Sistema de Gestión de Inventario y Ventas con MongoDB

Descripción

Se desarrollará una base de datos documental para administrar los productos y las ventas de una tienda comercial.

El sistema permitirá registrar artículos, consultar sus características, actualizar existencias, registrar ventas y obtener información relevante para la administración del negocio.

Modelo NoSQL propuesto

Utilizaremos dos colecciones principales.

Colección: productos

25 documentos

CAMPOS

_id: ObjectId
sku: String
nombre: String
categoria: String
precio_centavos: Number
stock: Number
proveedor: Object
activo: Boolean

Colección: ventas

10 documentos

CAMPOS

_id: ObjectId
numero_venta: String
fecha: Date
cliente: Object
productos: Array
total_centavos: Number
estado: String

Total inicial



35

Documentos distribuidos en dos colecciones.

Figura 1. Propuesta inicial del sistema y de las colecciones productos y ventas.

6.2 Generación asistida de operaciones CRUD

También se solicitó a la IA una guía para crear el archivo encargado de demostrar `insertOne()`, `findOne()`, `updateOne()` y `deleteOne()`. El código resultante se revisó y se fortaleció con comprobaciones de resultados, referencias existentes y validación del estado final.



Figura 2. Interacción con la IA para construir la demostración CRUD.

7. Análisis crítico y correcciones manuales

La respuesta de la IA fue útil para estructurar rápidamente una solución inicial, pero no cubrió por sí sola todos los aspectos necesarios para una implementación consistente. La revisión humana permitió detectar y corregir los siguientes puntos:

Aspecto	Propuesta inicial de IA	Revisión o corrección manual	Justificación
Cantidad de colecciones	Dos colecciones: productos y ventas.	Se añadieron <code>categorias</code> y <code>proveedores</code> , para un total de cuatro.	Evita repetir nombres y datos de catálogos en cada producto y permite administrarlos de forma independiente.
Volumen inicial	25 productos y 10 ventas: 35 documentos.	Se incorporaron 5 categorías y 5 proveedores: 45 documentos.	La solución final representa el dominio completo y supera el mínimo de 20 registros.
Categoría y proveedor	Campos simples o un objeto genérico dentro del producto.	Se utilizaron referencias <code>ObjectId</code> mediante <code>categoria_id</code> y <code>proveedor_id</code> .	Mejora la consistencia y permite reutilizar datos maestros.
Valores monetarios	Tipo numérico genérico.	Se almacenaron <code>precio_centavos</code> , <code>subtotal_centavos</code> y <code>total_centavos</code> con <code>NumberLong</code> .	Evita errores de precisión propios de los números decimales en cálculos monetarios.
Validación	La propuesta visual no definía reglas detalladas.	Se diseñaron esquemas <code>\$jsonSchema</code> , campos obligatorios, mínimos, tipos BSON, patrón de correo y estados permitidos.	Rechaza documentos incompletos o con tipos incorrectos.
Historial de ventas	Arreglo de productos sin estrategia histórica explícita.	Cada detalle conserva <code>producto_id</code> , SKU, nombre, cantidad, precio unitario y subtotal.	Una venta pasada no cambia cuando se modifica el catálogo actual.
CRUD	Operaciones básicas.	Se añadieron verificaciones de <code>acknowledged</code> , <code>matchedCount</code> , <code>modifiedCount</code> , <code>deletedCount</code> y conteo final.	Permite comprobar que cada operación produjo exactamente el efecto esperado.
Consultas	Comandos generales.	Se crearon filtros con <code>\$expr</code> , ordenamientos deterministas y una agregación con <code>\$match</code> , <code>\$unwind</code> , <code>\$group</code> , <code>\$sort</code> , <code>\$limit</code> y <code>\$project</code> .	Las consultas responden necesidades concretas del negocio y generan salidas verificables.
Rendimiento	Sin evidencia de optimización.	Se comparó el plan antes y después de un índice compuesto.	La mejora se sustenta con estadísticas reales, no únicamente con una recomendación teórica.

Una limitación de la prueba es que el conjunto de datos es pequeño; por ello, el tiempo registrado fue de 0 ms tanto antes como después del índice. Sin embargo, el cambio del plan de `COLLSCAN` a `IXSCAN` y la reducción de 25 a 5 documentos examinados sí demuestran que el motor realizó menos trabajo. En un conjunto de producción más grande, esta diferencia tendría mayor impacto.

8. Modelo implementado

La base de datos se denomina `inventario_ventas_db` y contiene cuatro colecciones.

Colección	Cantidad inicial	Función principal
<code>categorias</code>	5	Catálogo de clasificación de productos.
<code>proveedores</code>	5	Datos de las empresas que suministran productos.
<code>productos</code>	25	Catálogo operativo con precio, stock y referencias.
<code>ventas</code>	10	Transacciones confirmadas con cliente y detalle histórico embebido.
Total	45	Documentos cargados inicialmente.

8.1 Estructura de categorías

Cada categoría contiene `nombre`, `descripcion` y `activo`. El campo `activo` permite deshabilitar una categoría sin eliminarla físicamente.

8.2 Estructura de proveedores

Los proveedores almacenan `nombre`, `correo`, `telefono` y `activo`. El esquema verifica que el correo tenga una estructura básica válida.

8.3 Estructura de productos

Un producto incluye:

- `sku`: identificador comercial legible.
- `nombre`: descripción del producto.
- `categoria_id` y `proveedor_id`: referencias a catálogos.
- `precio_centavos`: valor monetario entero de 64 bits.
- `stock` y `stock_minimo`: existencias actuales y umbral de reposición.
- `activo`: estado lógico.
- `fecha_actualizacion`: fecha del último cambio.

8.4 Estructura de ventas

Cada venta contiene `numero_venta`, `fecha`, un objeto `cliente`, un arreglo `productos`, `total_centavos` y `estado`. El detalle embebido guarda la identificación y descripción histórica de cada producto junto con la cantidad, precio unitario y subtotal.

La relación lógica del modelo es la siguiente:

```
text categorias (1) —< productos >— (1) proveedores | | producto_id + copia  
histórica v ventas[]
```

9. Implementación y carga de datos

9.1 Entorno de ejecución

MongoDB 8.0 se ejecutó en un contenedor definido en `docker-compose.yml`. El puerto se publicó únicamente en `127.0.0.1:27017`, reduciendo la exposición de la instancia. Las credenciales se reciben mediante variables de entorno y los datos se conservan en el volumen `mongo_data`.

La conexión se realizó desde MongoDB Compass con un usuario administrativo.

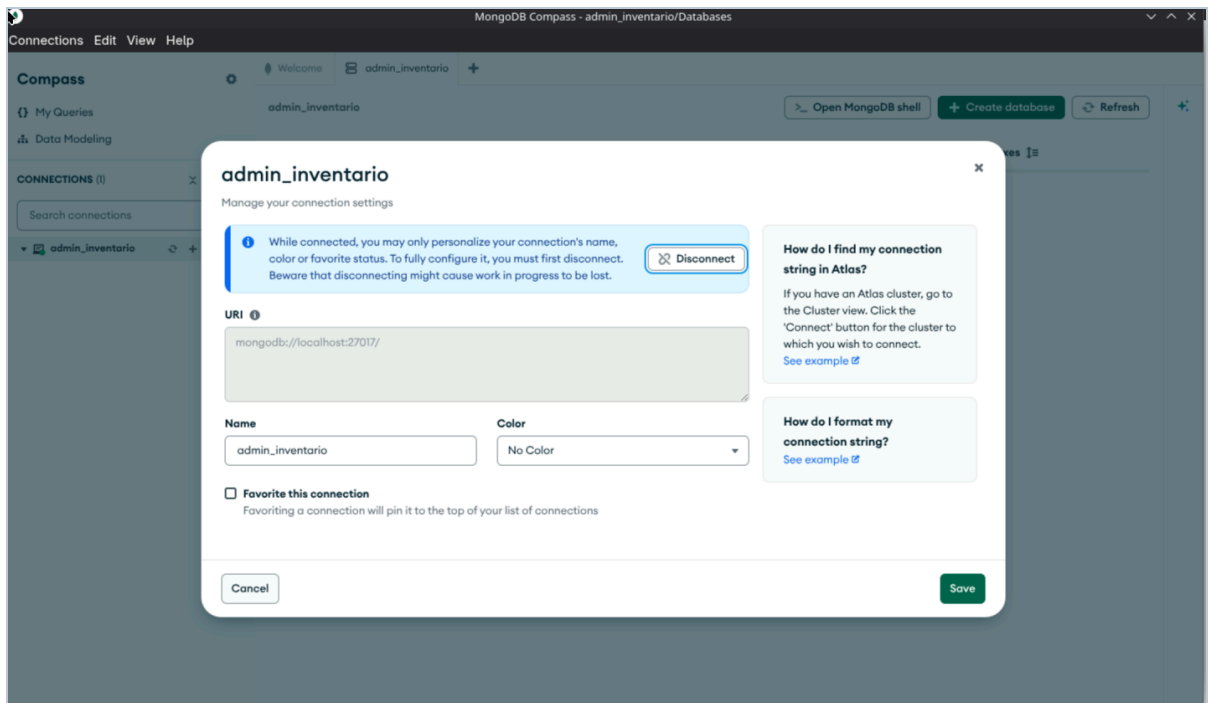


Figura 3. Configuración de la conexión local a MongoDB.

9.2 Definición de Docker Compose

La infraestructura se definió con el siguiente archivo `docker-compose.yml`. Se utiliza la imagen oficial de MongoDB 8.0, un contenedor con nombre identificable, un volumen persistente y un puerto accesible solamente desde el equipo local.

```
``yaml
services:
  mongodb:
    image: mongo:8.0
    container_name: nosql_inventario_mongodb
    restart: unless-stopped
    environment:
      MONGO_INITDB_ROOT_USERNAME: ${MONGO_ROOT_USERNAME}
      MONGO_INITDB_ROOT_PASSWORD: ${MONGO_ROOT_PASSWORD}
    ports:
      - "127.0.0.1:27017:27017"
    volumes:
      - mongo_data:/data/db

volumes:
  mongo_data:
```

El servicio se inicia con el siguiente comando desde la raíz del proyecto:

```
bash docker compose up -d
```

9.3 Creación de colecciones

El archivo `database/01-colecciones.js` define los esquemas de las colecciones y emplea validación estricta. La evidencia muestra la ejecución del script y la disponibilidad de las cuatro colecciones del modelo.

```
D ~/Documentos/GitHub/SBD2N_2S2026_201902672/NoSQL-Inventario-Ventas git main ?1 .....
docker compose exec mongodb sh -lc 'mongosh --quiet -u "$MONGO_INITDB_ROOT_USERNAME" -p "$MONGO_INITDB_ROOT_PASSWORD" --authenticationDatabases admin inventario_ventas_db --file /tmp/01-colecciones.js'
CREADA: categorias
CREADA: proveedores
ACTUALIZADA: productos
CREADA: ventas

Colecciones configuradas correctamente.
[
  'proveedores',
  'ventas',
  'categorias',
  'productos'
]
```

Figura 4. Ejecución del script de configuración de colecciones.

9.4 Inserción y verificación de datos

El archivo `database/02-datos-iniciales.js` carga 5 categorías, 5 proveedores, 25 productos y 10 ventas. Antes de insertar, verifica que las colecciones estén vacías para evitar duplicados. Durante la construcción de las ventas también comprueba la existencia del producto y la disponibilidad de stock.

```
D ~/Documentos/GitHub/SBD2N_2S2026_201902672/NoSQL-Inventario-Ventas git main ?1 .....
docker compose exec mongodb sh -lc 'mongosh --quiet -u "$MONGO_INITDB_ROOT_USERNAME" -p "$MONGO_INITDB_ROOT_PASSWORD" --authenticationDatabases admin inventario_ventas_db --file /tmp/02-datos-iniciales.js'
Insertando proveedores...
Insertando productos...
Insertando ventas...

===== RESULTADOS DE LA CARGA =====
{
  categorias: 5,
  proveedores: 5,
  productos: 25,
  ventas: 10
}
TOTAL DOCUMENTOS: 45
Carga inicial completada correctamente.
```

Figura 5. Carga inicial completada con 45 documentos.

MongoDB Compass confirmó las cantidades almacenadas en cada colección.

Collection name	Properties	Storage size	Data size	Documents	Avg. document size	Indexes	Total index size
categorias	-	20.48 kB	489.00 B	5	97.00 B	2	40.96 kB
productos	-	20.48 kB	5.20 kB	25	208.00 B	2	40.96 kB
proveedores	-	20.48 kB	574.00 B	5	114.00 B	2	40.96 kB
ventas	-	20.48 kB	5.31 kB	10	531.00 B	2	40.96 kB

Figura 6. Colecciones creadas y documentos persistidos.

10. Operaciones CRUD

El archivo `database/03-crud.js` utiliza el producto temporal con SKU `PRB-001`. Este enfoque permite demostrar las cuatro operaciones sin alterar permanentemente los 25 productos iniciales.

10.1 Create

`insertOne()` crea un producto de prueba con precio Q149.00, stock 10 y referencias existentes. Después se consulta el documento insertado para demostrar su persistencia.

```
~/Documentos/GitHub/SB02N_2S2026_201902672/NoSQL-Inventario-Ventas main ?1
docker compose exec mongod sh -lc 'mongosh --quiet -u "$MONGO_INITDB_ROOT_USERNAME" -p "$MONGO_INITDB_ROOT_PASSWORD" --authenticationDatabases admin inventario_ventas_db --file /tmp/03-crud.js'

===== VERIFICACIONES INICIALES =====
Productos iniciales: 25

===== CREATE =====
Producto insertado correctamente:
{
  acknowledged: true,
  insertedId: ObjectId('6ab6f06453470d18c563452d')
}
Documento creado:
{
  _id: ObjectId('6ab6f06453470d18c563452d'),
  sku: 'PRB-001',
  nombre: 'Producto de prueba CRUD',
  categoria_id: ObjectId('6ab6ef1e4d777da151b77690'),
  proveedor_id: ObjectId('6ab6ef1e4d777da151b77695'),
  precio_centavos: Long('14900'),
  stock: 10,
  stock_minimo: 3,
  activo: true,
  fecha_actualizacion: ISODate('2026-09-25T22:06:28.727Z')
}
```

Figura 7. Inserción verificada del producto temporal.

10.2 Read

`findOne()` localiza el producto por SKU y aplica una proyección para devolver únicamente los campos relevantes.

```
===== READ =====
Producto encontrado:
{
  _id: ObjectId('6ab6f06453470d18c563452d')
}
```

Figura 8. Lectura del producto temporal por su SKU.

10.3 Update

`updateOne()` cambia el precio de Q149.00 a Q139.00 mediante `$set`, incrementa el stock de 10 a 15 mediante `$inc` y actualiza la fecha de modificación. El resultado confirma un documento encontrado y uno modificado.

```

===== UPDATE =====
Resultado de la actualización:
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
Documento actualizado:
{
  _id: ObjectId('6ab6f06453470d18c563452d'),
  sku: 'PRB-001',
  nombre: 'Producto de prueba CRUD',
  categoria_id: ObjectId('6ab6ef1e4d777da151b77690'),
  proveedor_id: ObjectId('6ab6ef1e4d777da151b77695'),
  precio_centavos: Long('13900'),
  stock: 15,
  stock_minimo: 3,
  activo: true,
  fecha_actualizacion: ISODate('2026-09-25T22:06:28.743Z')
}

```

Figura 9. Actualización del precio y del inventario.

10.4 Delete

`deleteOne()` elimina únicamente el producto temporal. La consulta posterior devuelve `null` y el conteo final regresa a 25 productos, por lo que la prueba no deja residuos.

```

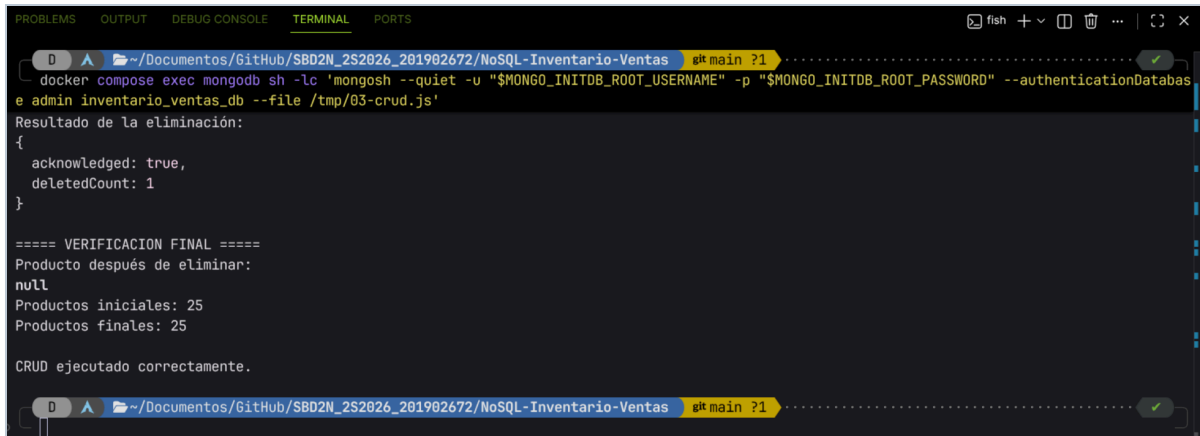
===== DELETE =====
Resultado de la eliminación:
{
  acknowledged: true,
  deletedCount: 1
}

===== VERIFICACION FINAL =====
Producto después de eliminar:
null
Productos iniciales: 25
Productos finales: 25

CRUD ejecutado correctamente.

```

Figura 10. Eliminación y verificación final del producto temporal.



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
~/Documentos/GitHub/SB02N_252026_201902672/NoSQL-Inventario-Ventas git main ?1
docker compose exec mongodb sh -lc 'mongosh --quiet -u "$MONGO_INITDB_ROOT_USERNAME" -p "$MONGO_INITDB_ROOT_PASSWORD" --authenticationDatabase admin inventario_ventas_db --file /tmp/03-crud.js'
Resultado de la eliminación:
{
  acknowledged: true,
  deletedCount: 1
}

==== VERIFICACION FINAL ====
Producto después de eliminar:
null
Productos iniciales: 25
Productos finales: 25

CRUD ejecutado correctamente.
```

Figura 11. Confirmación de que el CRUD finalizó correctamente.

11. Consultas relevantes

11.1 Ejecución de las consultas

Las consultas se definieron en `database/04-consultas.js`. Para ejecutarlas dentro del contenedor, primero se copió el archivo al directorio temporal de MongoDB y luego se invocó `mongosh` con las credenciales almacenadas en las variables de entorno del servicio.

```
```bash
docker cp database/04-consultas.js \
nosql_inventario_mongodb:/tmp/04-consultas.js

docker compose exec mongodb sh -lc \
'mongosh --quiet \
-u "$MONGO_INITDB_ROOT_USERNAME" \
-p "$MONGO_INITDB_ROOT_PASSWORD" \
--authenticationDatabase admin \
inventario_ventas_db \
--file /tmp/04-consultas.js'
```
```

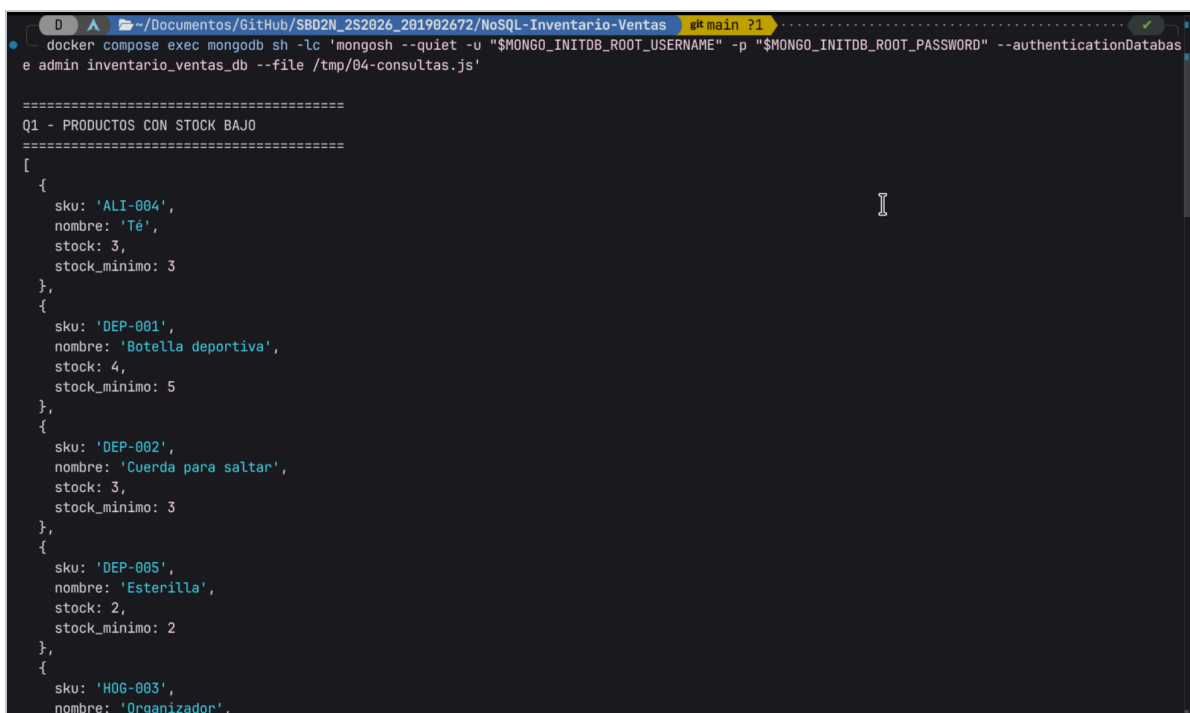
El archivo ejecuta las tres consultas de forma consecutiva y al final imprime un resumen con las cantidades obtenidas. Las Figuras 12 a 15 muestran la salida de esa ejecución.

11.2 Q1 — Productos con stock bajo

La primera consulta localiza productos activos cuyo `stock` es menor o igual que `stock_minimo`. Se utiliza `$expr` porque la comparación se realiza entre dos campos del mismo documento.

```
javascript db.productos.find({ activo: true, $expr: { $lte: ["$stock", "$stock_minimo"] } })
```

El resultado identificó **8 productos** que requieren atención de reposición. Esta consulta apoya las decisiones de compra y previene faltantes.



```
=====
Q1 - PRODUCTOS CON STOCK BAJO
=====
[
  {
    sku: 'ALI-004',
    nombre: 'Té',
    stock: 3,
    stock_minimo: 3
  },
  {
    sku: 'DEP-001',
    nombre: 'Botella deportiva',
    stock: 4,
    stock_minimo: 5
  },
  {
    sku: 'DEP-002',
    nombre: 'Cuerda para saltar',
    stock: 3,
    stock_minimo: 3
  },
  {
    sku: 'DEP-005',
    nombre: 'Esterilla',
    stock: 2,
    stock_minimo: 2
  },
  {
    sku: 'HOG-003',
    nombre: 'Organizador',
    stock: 2,
    stock_minimo: 2
  }
]
```

Figura 12. Q1: ocho productos con existencias iguales o inferiores al mínimo.

11.3 Q2 — Productos por categoría y precio

La segunda consulta obtiene la categoría Tecnología, filtra sus productos activos y los ordena por precio ascendente y SKU. El SKU actúa como criterio secundario para producir un orden determinista cuando existen precios iguales.

```
javascript db.productos.find({ categoria_id: categoria._id, activo: true }).sort({ precio_centavos: 1, sku: 1 })
```

Se obtuvieron **5 productos** de la categoría Tecnología. La consulta permite revisar el catálogo de una línea específica y comparar precios.

```
=====
Q2 - PRODUCTOS POR CATEGORIA Y PRECIO
=====
Categoría: Tecnología
[
  {
    sku: 'TEC-004',
    nombre: 'Memoria USB 64GB',
    precio_centavos: Long('6500'),
    stock: 4
  },
  {
    sku: 'TEC-001',
    nombre: 'Mouse inalámbrico',
    precio_centavos: Long('12999'),
    stock: 13
  },
  {
    sku: 'TEC-002',
    nombre: 'Teclado USB',
    precio_centavos: Long('19900'),
    stock: 10
  },
  {
    sku: 'TEC-003',
    nombre: 'Audífonos',
    precio_centavos: Long('24900'),
    stock: 3
  },
  {
    sku: 'TEC-005',
    nombre: 'Webcam HD',
    stock: 1
  }
]
```

Figura 13. Q2: productos tecnológicos ordenados por precio.

11.4 Q3 — Cinco productos más vendidos

La tercera consulta usa el framework de agregación. Primero conserva únicamente ventas confirmadas; después separa los elementos del arreglo con `$unwind`, agrupa por producto, suma las unidades y devuelve los primeros cinco registros.

```
javascript db.ventas.aggregate([ { $match: { estado: "CONFIRMADA" } }, { $unwind:
"$productos" }, { $group: { _id: "$productos.producto_id", sku: { $first:
"$productos.sku" }, nombre: { $first: "$productos.nombre" }, unidades_vendidas: {
$sum: "$productos.cantidad" } }}, { $sort: { unidades_vendidas: -1, sku: 1 } }, {
$limit: 5 }, { $project: { _id: 0, sku: 1, nombre: 1, unidades_vendidas: 1 } } ])
```

El ranking obtenido fue:

| Posición | SKU | Producto | Unidades vendidas |
|----------|---------|------------------------|-------------------|
| 1 | PAP-001 | Cuaderno universitario | 5 |
| 2 | PAP-002 | Bolígrafo azul | 5 |
| 3 | TEC-001 | Mouse inalámbrico | 5 |
| 4 | ALI-002 | Galletas | 4 |
| 5 | DEP-001 | Botella deportiva | 4 |

```
Q3 - TOP 5 PRODUCTOS MAS VENDIDOS
=====
[
  {
    sku: 'PAP-001',
    nombre: 'Cuaderno universitario',
    unidades_vendidas: 5
  },
  {
    sku: 'PAP-002',
    nombre: 'Bolígrafo azul',
    unidades_vendidas: 5
  },
  {
    sku: 'TEC-001',
    nombre: 'Mouse inalámbrico',
    unidades_vendidas: 5
  },
  {
    sku: 'ALI-002',
    nombre: 'Galletas',
    unidades_vendidas: 4
  },
  {
    sku: 'DEP-001',
    nombre: 'Botella deportiva',
    unidades_vendidas: 4
  }
]
Productos en el ranking: 5
=====
```

Figura 14. Q3: clasificación de productos por unidades vendidas.

El script consolidó los resultados de las tres consultas y confirmó su ejecución correcta.

```
=====
RESUMEN DE CONSULTAS
=====
{
  productos_stock_bajo: 8,
  productos_categoria: 5,
  productos_ranking: 5
}

Consultas ejecutadas correctamente.
```

Figura 15. Resumen: 8 productos con stock bajo, 5 por categoría y 5 en el ranking.

12. Optimización mediante índices

La consulta Q2 filtra por `categoria_id` y `activo`, y luego ordena por `precio_centavos` y `sku`. Por esa razón se diseñó el índice compuesto:

```
javascript db.productos.createIndex( { categoria_id: 1, activo: 1, precio_centavos: 1, sku: 1 }, { name: "idx_categoria_activo_precio_sku" } )
```

El orden de los campos sigue el patrón de igualdad antes de ordenamiento: primero aparecen los campos usados con coincidencia exacta y después los campos del `sort`.

12.1 Plan anterior

Antes del índice, MongoDB utilizó `COLLSCAN`, examinó los **25 documentos** de la colección, no examinó claves y devolvió 5 resultados.

```

===== ANTES DE OPTIMIZAR =====
{
  documentos_devueltos: 5,
  documentos_examinados: 25,
  claves_examinadas: 0,
  tiempo_ms: 0
}
Plan de ejecución anterior:
{
  isCached: false,
  stage: 'SORT',
  sortPattern: {
    precio_centavos: 1,
    sku: 1
  },
  memLimit: 104857600,
  type: 'simple',
  inputStage: {
    stage: 'COLLSCAN',
    filter: {
      '$and': [
        {
          activo: {
            '$eq': true
          }
        }
      ]
    }
  },
  {

```

Figura 16. Plan anterior: escaneo completo de la colección.

12.2 Plan optimizado

Después de crear el índice, el plan incorporó **IXSCAN**; examinó **5 claves** y **5 documentos** para devolver los mismos 5 resultados. Además, el índice satisfizo el orden solicitado.

```

===== CREANDO INDICE =====
Índice creado: idx_categoria_activo_precio_sku

===== DESPUES DE OPTIMIZAR =====
{
  documentos_devueltos: 5,
  documentos_examinados: 5,
  claves_examinadas: 5,
  tiempo_ms: 1
}
Plan de ejecución posterior:
{
  isCached: false,
  stage: 'FETCH',
  inputStage: {
    stage: 'IXSCAN',
    keyPattern: {
      categoria_id: 1,
      activo: 1,
      precio_centavos: 1,
      sku: 1
    },
    indexName: 'idx_categoria_activo_precio_sku',
    isMultiKey: false,
    multiKeyPaths: {
      categoria_id: [],
      activo: [],

```

Figura 17. Plan optimizado con escaneo del índice compuesto.

```

===== INDICES DISPONIBLES =====
{
  {
    v: 2,
    key: {
      _id: 1
    },
    name: '_id_'
  },
  {
    v: 2,
    key: {
      sku: 1
    },
    name: 'sku_1',
    unique: true
  },
  {
    v: 2,
    key: {
      categoria_id: 1,
      activo: 1,
      precio_centavos: 1,
      sku: 1
    },
    name: 'idx_categoria_activo_precio_sku'
  }
}

```

Figura 18. Verificación del índice compuesto disponible en productos.

| Métrica | Antes | Después |
|---------------------------------|----------|---------|
| Documentos devueltos | 5 | 5 |
| Documentos examinados | 25 | 5 |
| Claves examinadas | 0 | 5 |
| Etapas principales observadas | COLLSCAN | IXSCAN |
| Tiempo en el conjunto de prueba | 0 ms | 0 ms |

El índice redujo en un **80 %** los documentos examinados: $(25 - 5) / 25 \times 100$. Aunque el tiempo no varió por el tamaño reducido de la muestra, el plan posterior es más escalable.

13. Conclusiones

1. MongoDB permitió representar de forma natural las ventas mediante documentos con objetos y arreglos embebidos, mientras que las referencias conservaron separados los catálogos de categorías y proveedores.
2. La solución implementada cumplió el mínimo de registros al cargar 45 documentos y demostró operaciones CRUD completas sin modificar de forma permanente el conjunto inicial.
3. Las tres consultas desarrolladas aportan información accionable: alertas de reposición, exploración de productos por categoría y clasificación de los productos más vendidos.

4. El uso de enteros en centavos, validaciones de esquema y verificaciones explícitas fortaleció la integridad de los datos respecto de la propuesta inicial.
5. La IA fue útil para acelerar el modelado y la generación de comandos, pero la revisión humana fue necesaria para adaptar, probar, corregir y justificar la solución.
6. La medición con `explain("executionStats")` permitió comprobar que el índice compuesto redujo los documentos examinados y reemplazó el escaneo completo por un escaneo de índice.

14. Reflexión individual

Aunque la IA generativa puede acelerar la construcción de soluciones, no reemplaza la necesidad de conocimientos humanos. La revisión crítica, la comprensión del dominio y la validación técnica son esenciales para garantizar que el resultado final sea correcto, eficiente y útil para el negocio.

14.1 ¿La IA ayudó realmente al desarrollo?

Si, la IA proporcionó un punto de partida rápido para el modelo y los comandos CRUD. Sin embargo, su propuesta inicial no cumplía con todos los requisitos del caso de estudio, por lo que fue necesario realizar ajustes manuales significativos.

14.2 ¿Qué errores detectó?

La IA cometió varios errores, entre ellos:

- Proponer solo dos colecciones en lugar de cuatro, lo que habría generado duplicación de datos y dificultades de mantenimiento.
- No utilizar referencias para categorías y proveedores, lo que habría afectado la consistencia de los datos.
- Esquemas de validación incompletos, lo que podría haber permitido la inserción de documentos inválidos.
- No conservar datos históricos en las ventas, lo que habría dificultado el análisis de transacciones pasadas.
- Sugerir tipos de datos inadecuados para valores monetarios, lo que podría haber causado problemas de precisión en cálculos financieros.

14.3 ¿Qué conocimientos humanos fueron necesarios para corregir la solución?

- Comprender el dominio del negocio para identificar las entidades y relaciones correctas.
- Conocer las practicas de modelado de datos en MongoDB, incluyendo el uso de referencias y datos embebidos.
- Aplicar validaciones de esquema para garantizar la integridad de los datos.
- Diseñar consultas eficientes y comprender cómo optimizar el rendimiento mediante índices.

- Evaluar críticamente las recomendaciones de la IA y decidir cuándo aceptar, modificar o rechazar sus sugerencias.

14.4 ¿Qué riesgos existen al depender totalmente de IA?

- La IA puede generar soluciones que parecen correctas pero que contienen errores lógicos o técnicos.
- Puede no comprender completamente el contexto del negocio, lo que lleva a decisiones de diseño inapropiadas.
- La falta de revisión humana puede resultar en problemas de integridad de datos, rendimiento deficiente y dificultades de mantenimiento a largo plazo.
- La dependencia excesiva de la IA puede limitar el desarrollo de habilidades críticas y de resolución de problemas en los desarrolladores humanos.

15. Referencias técnicas

- MongoDB, *Documentación oficial: Data Modeling*.
<https://www.mongodb.com/docs/manual/data-modeling/>
 - MongoDB, *Documentación oficial: Schema Validation*.
<https://www.mongodb.com/docs/manual/core/schema-validation/>
 - MongoDB, *Documentación oficial: Aggregation Operations*.
<https://www.mongodb.com/docs/manual/aggregation/>
 - MongoDB, *Documentación oficial: explain Results*.
<https://www.mongodb.com/docs/manual/reference/explain-results/>
 - OpenAI, ChatGPT, herramienta de IA generativa utilizada como apoyo para la propuesta de modelo y comandos CRUD.
-